

## Project work - Travel demand forecasting using SARIMA

ISM-E1005 Forecasting Methods in Business Analytics

Milla Jylänki 927031

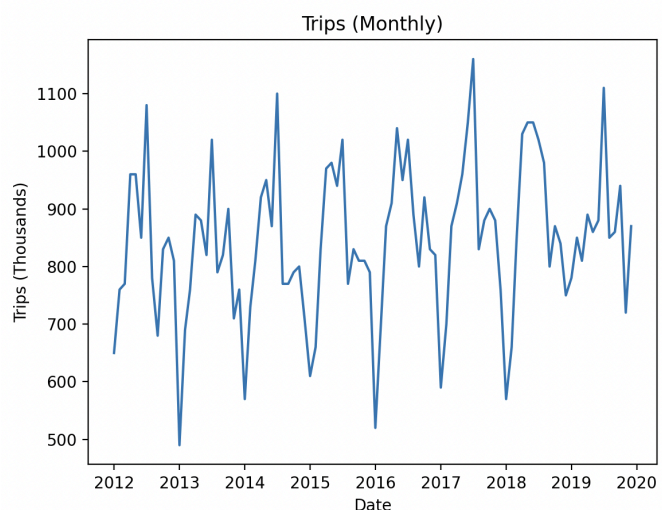
The research question I chose is how the Box-Jenkins method of applying a SARIMA model can be applied to the monthly numbers of travel abroad by Finnish citizens. For the data to study, I chose a dataset from Statistics Finland depicting monthly trips abroad taken by Finnish citizens from the database **13q6 -- Trips abroad, monthly, 2012M01-2025M11**. The dependent variable is the monthly number of trips abroad per calendar month, expressed in counts of thousands. The chosen sample covers monthly volumes from January 2012 to December 2019. I chose to focus on the pre-pandemic period to avoid structural breaks caused by COVID-19.

The data can be accessed [here](#). I selected the variables:

Information - Trips, thousand trips

Month - 2012M01 -> 2019M12

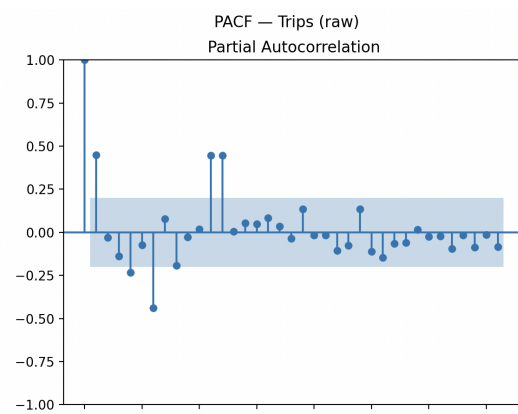
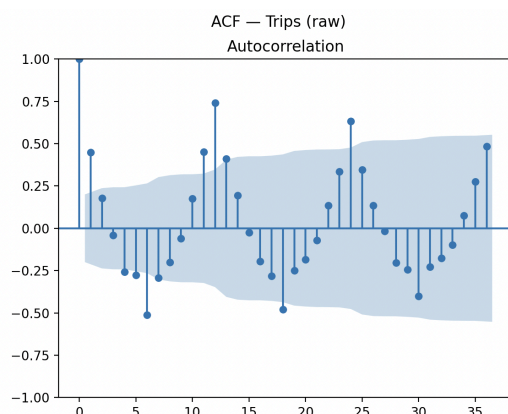
I used Python to display a time series plot to view the trend and seasonality of the raw data. From the plot on the right, we can see a slight upward trend over time. Monthly passenger volumes are shown to range from 500 to 1100 thousand, and the average monthly volume is around 700-900 thousand. The values are noticeably higher during summer months and lower during winter months, which indicates an annual seasonal pattern.



The dataset chosen shows a clear trend and seasonal dynamics, which makes it an excellent candidate for using the Box-Jenkins methodology of fitting an ARIMA model for a time series for short-term forecasting.

### 1. Model identification

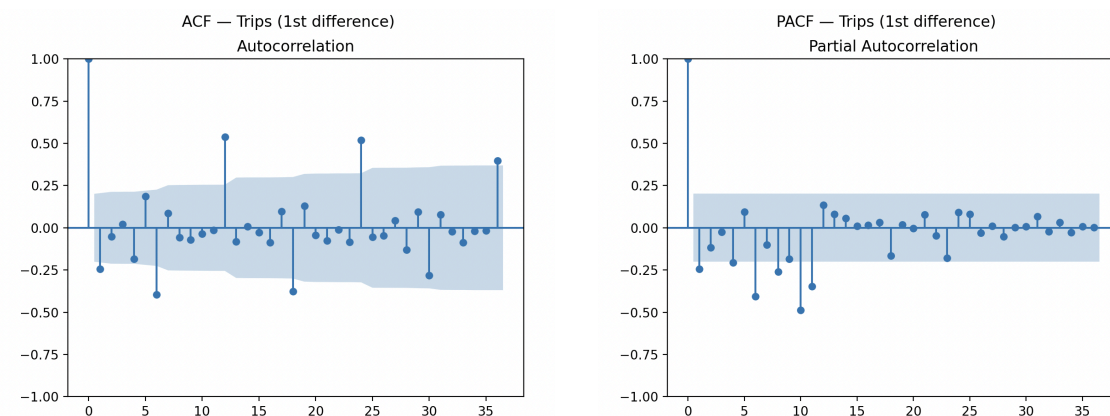
To be able to fit a forecasting model to the series, we must first determine the stationarity. I first plotted the autocorrelation and partial autocorrelation functions of the raw data. I chose the maximum number of lags as 36, or three years of monthly lags, to have enough data to view seasonality.



The ACF of the raw data shows us that the autocorrelation does not drop to zero instantly; instead, the first two lags are positive. This could indicate that consecutive monthly trips are related. A slower decay indicates non-stationarity, usually caused by a trend. There are also clear seasonal spikes at lags 12, 24, and 36, which tells us there is a seasonal pattern (yearly peaks or drops in the number of travellers). The presence of a trend and a seasonal component shows us that we should estimate the correct model using first-order and seasonal differencing.

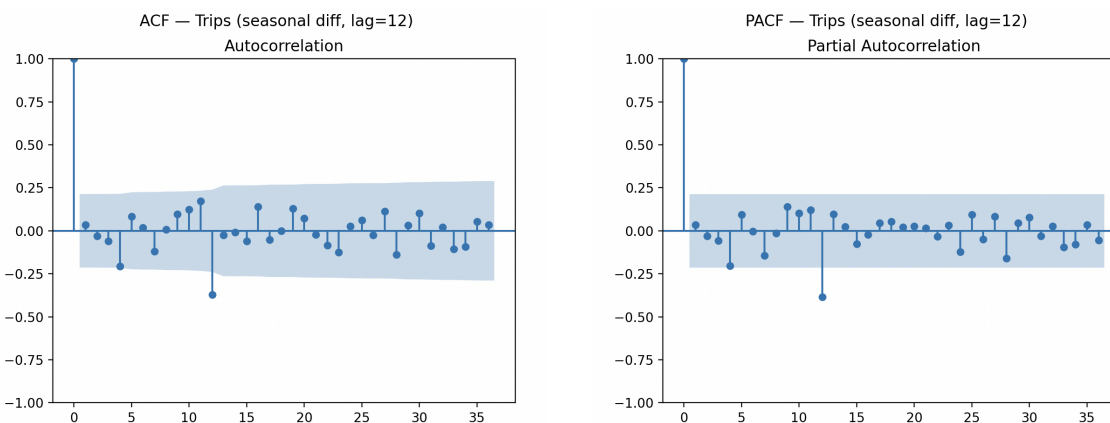
The PACF of the raw data shows us a strong spike at lag 1, and spikes around lag 12, which again indicates seasonality. However, because the series seems to be non-stationary, we cannot use just the ACF and PACF of the raw data for model identification.

Because we suspect a trend, I plotted the ACF and PACF of the data differenced to the first order.



After differencing the data, we can see that the ACF no longer shows the slow decay of the raw series, which indicates that the trend has been effectively removed, and we do not need differencing with a higher order. This means that the order of integrability  $h = 1$ . The large spikes at lags 12, 24, and 36 indicate remaining seasonal dependence, which means seasonal differencing is still required. On the PACF, most short lags are insignificant. The negative value at lag 1 does not yet provide enough evidence for a pure AR or MA structure.

Next, I applied seasonal differencing to help determine the seasonal components of the SARIMA model. The lag 12 was chosen to remove the yearly pattern.

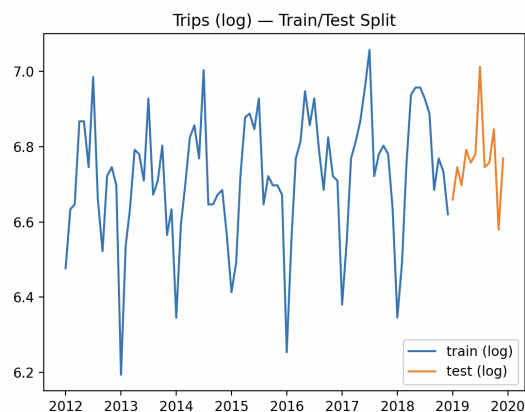


The positive spikes in the ACF at lags 12, 24, and 36 are gone, indicating that the annual seasonality has been successfully removed. One seasonal difference is sufficient to remove it, so the order of seasonal integrability is  $H = 1$ . The strong negative spike at lag 12, followed by a rapid cutoff, implies that a seasonal moving average term ( $Q = 1$ ) can be included in the model to capture remaining seasonal dependence.

## 2. Model estimation

Since the variance of passenger volumes appears to increase with the level of the series, the natural logarithm of the variable can be used to stabilize variance and account for seasonal patterns. I divided the sample into a training and a test sample based on time - the training sample consists of the first 36 months, and the test sample contains the final 12 months of data.

The plot below displays the log-transformed series split into the training sample and the test sample.



Based on the autocorrelation analysis and the stationarity tests, I modeled the series using the SARIMA model planned in the identification stage. The data was differenced to the first order, and seasonally differenced once, so the orders of integrality ( $h, H$ ) were both set to 1. Based on the seasonal differencing, the seasonal moving average component ( $Q$ ) was set to 1. Thus, the final model is

$$SARIMA(0, 1, 0)(0, 1, 1)_{12}$$

I estimated the model again using maximum likelihood via the SARIMAX framework in Python. Below is the model summary - the Ljung-Box test probability in this framework is now 0.00, which still means the null hypothesis of no residual autocorrelation is rejected, but I'll go into more detail on that in the next section.

```

SARIMAX Results
=====
Dep. Variable:          Trips    No. Observations:          84
Model:                 SARIMAX(0, 1, 0)x(0, 1, [1], 12)    Log Likelihood          51.428
Date:                  Sat, 14 Feb 2026    AIC                  -98.855
Time:                  14:16:08    BIC                  -94.734
Sample:                01-01-2012    HQIC                 -97.250
                    - 12-01-2018
Covariance Type:       opg
=====
coef    std err          z      P>|z|    [0.025    0.975]

```

```
-----
ma.S.L12      -0.6337      0.176      -3.604      0.000      -0.978      -0.289
sigma2         0.0096      0.002      4.714      0.000      0.006      0.014
=====
Ljung-Box (L1) (Q):                27.86      Jarque-Bera (JB):                0.69
Prob(Q):                           0.00      Prob(JB):                        0.71
Heteroskedasticity (H):             1.10      Skew:                            0.16
Prob(H) (two-sided):                0.83      Kurtosis:                       2.58
=====
```

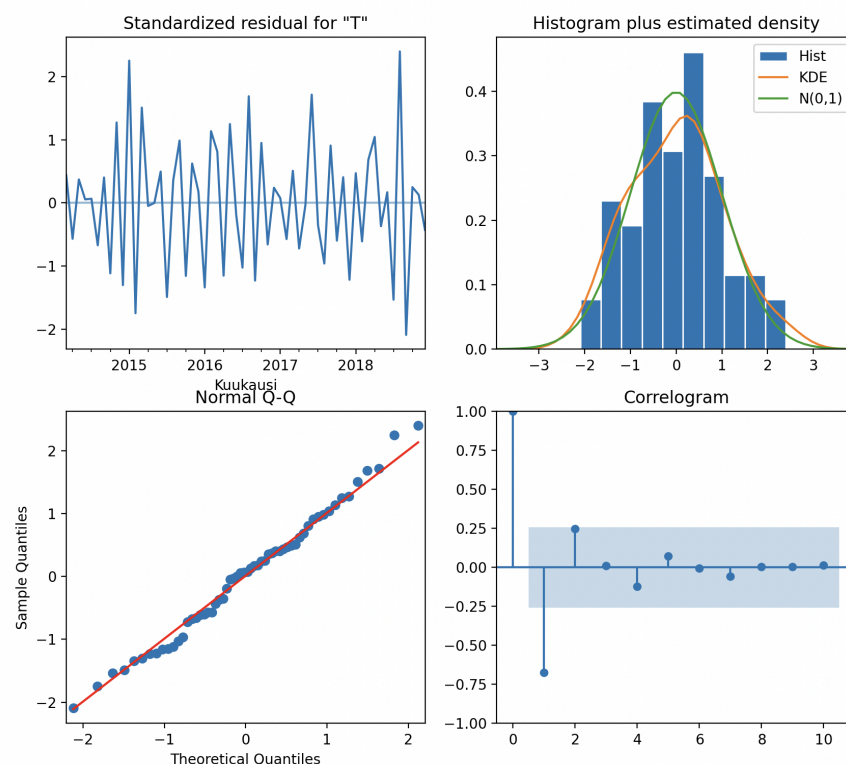
Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

### 3. Diagnostics

For the model to be a suitable fit, the residuals or forecast errors should behave like white noise, i.e., have a mean close to 0, constant variance, no remaining autocorrelation, and a roughly normal distribution. To test the behavior of the residuals, I used plot diagnostics on Python to detect the variance, mean, distribution, and the ACF of residuals, and the Ljung-Box test to check whether the residuals have autocorrelation.

Below are the plots produced by the plot diagnostics code.



The standardized residuals fluctuate quite randomly around zero and do not show a trend or seasonality. The histogram is quite bell-shaped and centered around zero, although skewness is visible. The Q-Q plot shows that the points are mostly close to the reference line, with minor deviations in the tails, indicating approximate

normality. The correlogram shows that the first two correlations are outside of the 95% confidence bounds, with a strong negative value at the first correlation, which tells us there is some remaining autocorrelation in the residuals. All in all, the residual diagnostics suggest that the model captures most of the systematic structure of the data, though minor remaining autocorrelation suggests that the model is not quite perfectly specified.

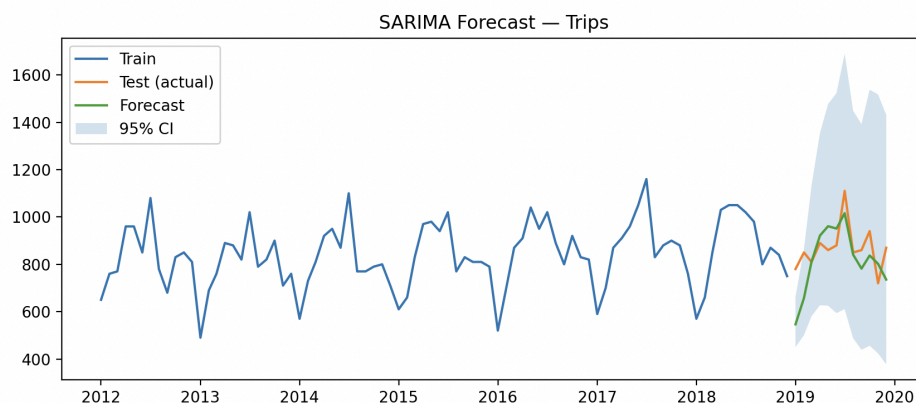
The Ljung-Box test produced these results:

|    | lb_stat   | lb_pvalue |
|----|-----------|-----------|
| 12 | 24.108233 | 0.019662  |
| 24 | 24.393676 | 0.439280  |

The null hypothesis of the Ljung-Box test is that residuals are not autocorrelated up to lag  $k$  (in this instance, 12 or 24). At lag 12, the p-value is below 0.05, which means the null hypothesis is rejected and indicates there is remaining autocorrelation. However, at lag 24, the p-value is well above 0.05, which means there is no autocorrelation. This suggests that with this model, short-term autocorrelation may remain, but dependence weakens at longer lags.

#### 4. Forecasting and evaluation

Finally, I used the  $SARIMA(0, 1, 0)(0, 1, 1)_{12}$  model to produce a 12-month forecast for the test period. I also calculated the forecast accuracy metrics of Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE). Below is the forecast plot.



The forecast follows the seasonal pattern, with higher passenger volumes in the summer and lower volumes in the winter. The 95% prediction intervals are very wide - although the actual values fall within the prediction intervals, their width indicates significant forecast uncertainty.

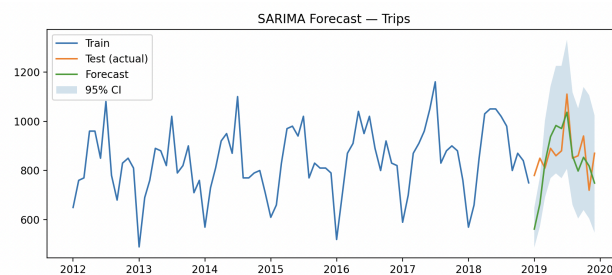
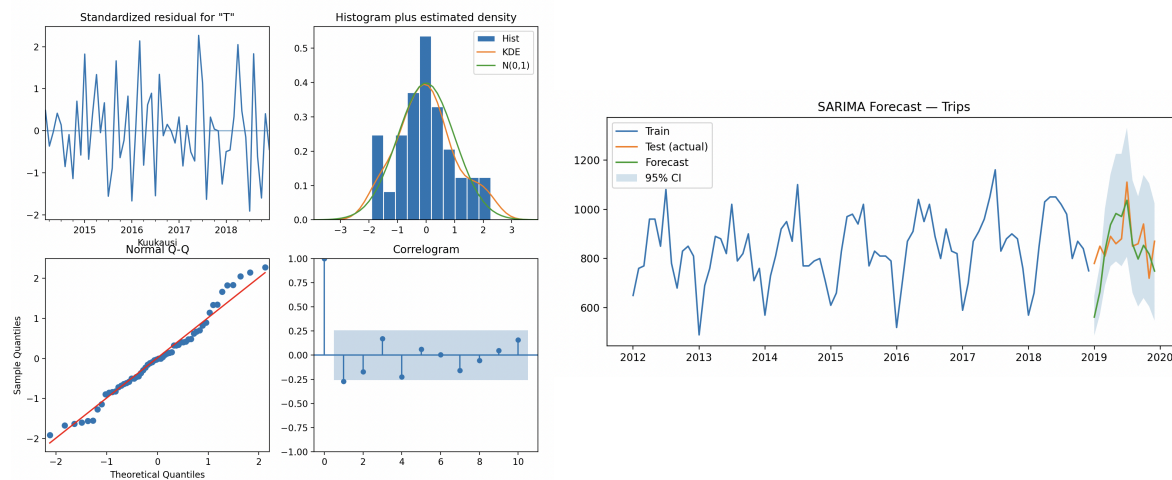
The calculations of the metrics produced these results:

MAE : 94.268  
 RMSE: 114.912  
 MAPE: 11.04%

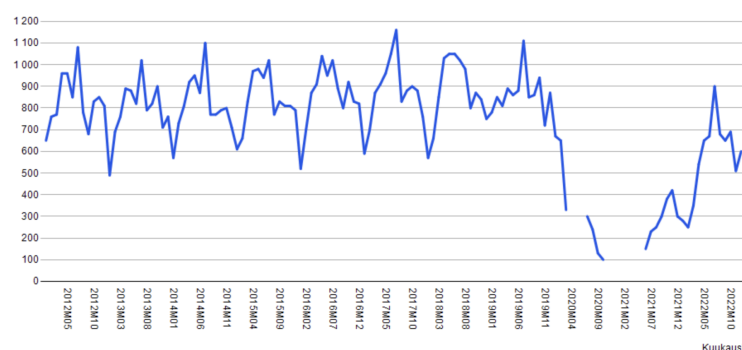
The MAE of 94.3 indicates that monthly forecasts deviate from actual values by about 94 units on average, and the RMSE suggests some larger forecast errors. The MAPE implies that forecasts differ from actual values by

11.04% on average. While the model captures the seasonal pattern well, this amount of forecast errors suggests that improvements are needed.

As seen by the residuals, the chosen model is not quite the perfect fit for this data set. For a more accurate fit, the model may need additional components; for example, an autoregressive or seasonal autoregressive component could increase accuracy. I tested out  $SARIMA(1, 1, 0)(0, 1, 1)_{12}$ , and the residual tests and forecast improved slightly, as shown below (the histogram shows less skewness, the correlogram has most correlations within confidence bounds, and the 95% prediction intervals are smaller).



Limitations of the study should also be acknowledged. The model is univariate and does not account for any external factors, which in this case could be economic conditions, fuel prices, or policy changes. The forecast assumes that the patterns of upward trend and seasonality will continue, while structural changes, such as economic crises or pandemics, can affect the situation significantly and lead to large forecast errors. This can be seen from the actual data that includes years after 2019:



While this research shows that the Box-Jenkins method could be used in this context to apply a SARIMA model to forecast accurately in the short-term, for long-term analysis, adding explanatory variables would be more suitable.

Here is the code used to compute results:

```
# ----- Section 0 -----

import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

#Opening the file from computer and changing titles to more suitable

df = pd.read_csv(
    "trips-abroad.txt",
    sep="\t",
    encoding="latin1"
)

df['Kuukausi'] = pd.to_datetime(df['Kuukausi'], format='%YM%m')
ts = (df.set_index('Kuukausi').asfreq("MS"))
print(ts.columns)
y = ts['Ulkomaanmatkat'].rename('Trips')

print(y.head())
print(y.index.min(), "→", y.index.max(), "| freq:", y.index.freqstr)

# Plotting the raw data

plt.figure()
plt.plot(y)
plt.title("Trips (Monthly)")
plt.xlabel("Date")
plt.ylabel("Trips (Thousands)")
ax = plt.gca()
ax.xaxis.set_major_locator(mdates.YearLocator())
plt.show()

# ----- Section 1 -----
# Model identification

import numpy as np
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# --- ACF / PACF on the raw series ---
max_lags = 36 # 3 years of monthly lags
```



```

fig = plot_acf(y, lags=max_lags)
fig.suptitle("ACF - Trips (raw)")
plt.show()

fig = plot_pacf(y, lags=max_lags, method="ywm")
fig.suptitle("PACF - Trips (raw)")
plt.show()

# --- ACF / PACF on first-differenced series ---
y_diff1 = y.diff().dropna()

fig = plot_acf(y_diff1, lags=max_lags)
fig.suptitle("ACF - Trips (1st difference)")
plt.show()

fig = plot_pacf(y_diff1, lags=max_lags, method="ywm")
fig.suptitle("PACF - Trips (1st difference)")
plt.show()

# --- Seasonal difference at lag 12 for monthly seasonality ---
y_seasdiff = y.diff(12).dropna()

fig = plot_acf(y_seasdiff, lags=max_lags)
fig.suptitle("ACF - Trips (seasonal diff, lag=12)")
plt.show()

fig = plot_pacf(y_seasdiff, lags=max_lags, method="ywm")
fig.suptitle("PACF - Trips (seasonal diff, lag=12)")
plt.show()

# ----- Section 2 -----
# Model estimation

# Use log to stabilize variance
y_log = np.log(y)

# Time-based split (last 12 months as test)
h = 12
train = y_log.iloc[:-h]
test = y_log.iloc[-h:]

print("Train:", train.index.min().date(), "→", train.index.max().date(), "| n=",
len(train))

```



```

print("Test :", test.index.min().date(), "→", test.index.max().date(), "| n=",
len(test))

plt.figure()
plt.plot(train, label="train (log)")
plt.plot(test, label="test (log)")
plt.title("Trips (log) – Train/Test Split")
ax = plt.gca()
ax.ticklabel_format(axis='y', style='plain')
ax.xaxis.set_major_locator(mdates.YearLocator())
plt.legend()
plt.show()

import statsmodels.api as sm
from statsmodels.stats.diagnostic import acorr_ljungbox

model = sm.tsa.SARIMAX(
    train,
    order=(0, 1, 0),
    seasonal_order=(0, 1, 1, 12),
    enforce_stationarity=False,
    enforce_invertibility=False
)

res = model.fit(dispatch=False)
print(res.summary())

# ----- Section 3 -----
# Diagnostics

# Built-in diagnostic plots: standardized residuals, QQ, ACF of residuals, etc.
res.plot_diagnostics(figsize=(10, 12))
plt.show()

# Ljung-Box test for residual autocorrelation (want p-values not tiny)
lb = acorr_ljungbox(res.resid.dropna(), lags=[12, 24], return_df=True)
print(lb)

# ----- Section 4 -----
# Forecasting and evaluation

```

```

import sklearn as sklearn
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Forecast h steps ahead
fc = res.get_forecast(steps=h)

fc_mean_log = fc.predicted_mean
fc_ci_log    = fc.conf_int()

# Back-transform to original scale
fc_mean = np.exp(fc_mean_log)
fc_lower = np.exp(fc_ci_log.iloc[:, 0])
fc_upper = np.exp(fc_ci_log.iloc[:, 1])

y_test_orig = np.exp(test)
y_train_orig = np.exp(train)

# Metrics on original scale
mae = mean_absolute_error(y_test_orig, fc_mean)
rmse = np.sqrt(mean_squared_error(y_test_orig, fc_mean))
mape = np.mean(np.abs((y_test_orig - fc_mean) / y_test_orig)) * 100

print(f"MAE : {mae:.3f}")
print(f"RMSE: {rmse:.3f}")
print(f"MAPE: {mape:.2f}%")

plt.figure(figsize=(10, 4))
plt.plot(y_train_orig, label="Train")
plt.plot(y_test_orig, label="Test (actual)")
plt.plot(fc_mean, label="Forecast")
ax = plt.gca()
ax.ticklabel_format(axis='y', style='plain')
plt.fill_between(fc_mean.index, fc_lower, fc_upper, alpha=0.2, label="95% CI")
plt.title("SARIMA Forecast - Trips")
plt.legend()
plt.show()

```